When Agents Fail: A Comprehensive Study of Bugs in LLM Agents with Automated Labeling

Niful Islam[1], Ragib Shahariar Ayon[2], Deepak George Thomas[3], Shibbir Ahmed[2], Mohammad Wardat[1,*]
[1]Oakland University, Rochester Hills, USA
[2]Texas State University, San Marcos, USA
[3]Tulane University, New Orleans, USA

arXiv:2601.15232v3 [cs.SE] 19 Aug 2026

***Abstract*—Large Language Models (LLMs) have revolutionized intelligent application development. While standalone LLMs cannot perform any actions, LLM agents address the limitation by integrating tools. However, debugging LLM agents is difficult and costly, as the field is still in its early stages and the community is underdeveloped. To understand the bugs encountered during agent development, we present the first comprehensive study of bug types, root causes, and effects in LLM agent-based software. We collected and analyzed 1,268 bug-related posts and code snippets from Stack Overflow, GitHub, and Hugging Face forums, focused on LLM agents built with seven widely used LLM frameworks as well as custom implementations. For a deeper analysis, we have also studied the component of the LLM agent where the bug occurred, along with the programming language and framework. This study also investigates the feasibility of automating bug identification. For that, we have built a *ReAct* agent named *BugReAct*, equipped with adequate external tools to determine whether it can detect and annotate the bugs in our dataset. According to our study, we found that *BugReAct* equipped with Gemini 2.5 Flash achieved a remarkable performance in annotating bug characteristics with an average cost of 0.01 USD per post/code snippet.**

***Index Terms*—Large Language Model, AI Agents, Bug Study**

I. Introduction

Large Language Model (LLM) agents are autonomous systems with human-like abilities such as autonomy, reasoning, and planning [1]. With recent advancements in LLMs, their adoption has grown rapidly across safety-critical domains including security [2], healthcare [3], robotics [4], and end-to-end software development [5], [6]. Like any software, LLM-based agents are susceptible to bugs, and understanding their types, root causes, and effects is essential for the software engineering community to mitigate them, a challenge compounded by the rapid proliferation of LLM agent frameworks.

While LLM agents are a form of software, they differ from traditional systems in three key ways. First, their reliance on black-box LLMs makes debugging more complex [7]. Second, they are nondeterministic, meaning the same input can produce different outputs depending on temperature settings [8], which makes bug reproduction more challenging. Third, the LLM agent ecosystem is still nascent. For example, LangChain, the most widely used framework, was only released in late 2022, and fewer experienced developers are available on community platforms to provide support [9]. These factors collectively motivate an extensive study of bugs in LLM agents.

*Corresponding author: wardat@oakland.edu

Recent work has examined defects in LLM agents and proposed taxonomies of agent-based bugs and errors [10], [11]. However, one effort focuses only on defect types, while the other primarily analyzes bugs in agent-building frameworks rather than the agents themselves. Additionally, while LLM agents have demonstrated exceptional abilities across multiple fields [12], [13], their effectiveness drops significantly when tasked with fixing issues within agentic systems. For example, the best-performing agent achieved only 4% accuracy [11], which highlights their currently limited self-healing capability.

To fill this gap, we present the first study of bugs in LLM agents across multiple platforms. Following Islam *et al.* [14], we characterize bugs by type, root cause, and effect, and identify the most affected agent component. We also analyze the distribution of bug types, root causes, and frameworks, and examine how bugs evolved over time. Additionally, we built a *BugReAct* agent that automatically labels bugs from online sources to assist developers in resolving them.

Building this study presented several challenges. Given the recent emergence of LLM agents, developers tend to seek help through conversational AI tools rather than conventional platforms, which reduces the quantity and quality of data available in traditional sources [14], [15]. Therefore, in addition to Stack Overflow and GitHub, we extended our search to Hugging Face forums for broader data collection. To mitigate biases from foundation models, *BugReAct* was developed using three distinct models, which prevents findings from depending on any single one. Finally, since the field evolves rapidly, encountered bugs were often novel, which posed a non-trivial challenge for both human and automated annotators. We release the dataset to assist developers in navigating this dynamic field. In summary, this paper makes the following main contributions:

- To the best of our knowledge, we conduct the first large-scale empirical study on LLM agent bugs. We collect bugs from Stack Overflow, GitHub, and Hugging Face across seven frameworks and organize into a taxonomy.
- We propose *BugReAct*, an LLM-based agent that automatically annotates bug-related posts by analyzing content, and invoking external tools.
- We evaluate *BugReAct*on 1,268 posts across three LLMs. It achieves bug characterization comparable to manual labeling and shows potential to support bug repair.

II. Related Work

*a) **Empirical Study on Bugs**:* There has been extensive research on bugs in data mining and information retrieval tools [16], deep learning systems [14], [15], attention-based models [17], deep reinforcement learning [18], and LLM libraries [19], [20], with studies focused on root causes, fault taxonomies, and bug patterns. While limited prior work has focused on LLM agents specifically, Xue *et al.* [21] and Zhang *et al.* [22] analyzed bugs across LLM agent frameworks. Pan *et al.* [23] analyzed execution traces to investigate failures in Multi-Agent Systems (MAS). Islam *et al.* [24] conducted a study analyzing bug-fix patterns in LLM agents. Lastly, Rahardja *et al.* [11] built a taxonomy of LLM agent issues from 201 GitHub issues across 18 repositories, 11 of which are agent frameworks such as CrewAI, and evaluated SoTA coding agents on agentic tasks: Agentless [25], AutoCodeRover [26], and SWE-agent [27] resolved only 3.33%, 1.33%, and 0.67% of agentic issues, versus 32.0%, 30.67%, and 18.33% on traditional tasks. Their taxonomy is built largely from framework and tool issues, while ours focuses on bugs that originate from the agents themselves.

*b) **Agentic AI for SE and SE for Agentic AI**:* In existing literature, agents have been used as a tool for fixing errors in software systems. For instance, Agentless [25], an LLM agent, is used for bug localization, repair, and patch validation. Similarly, AGDebugger [28] addresses multi-agent debugging through interactive features and real-time agent steering. RAFFLES [29] is used as an iterative Judge-Evaluator framework for automated fault attribution. Nonetheless, some research has evaluated bugs in agentic systems as well. Ning *et al.* [10] mined Stack Overflow posts on LLM agents and developed a static analysis approach for defect detection. Unlike their work, which focuses solely on defect types, our study examines bug types, root causes, effects, and responsible components, and relies on a *BugReAct* agent for automatic annotation. Deshpande *et al.* [30] proposed a taxonomy of agent errors and investigated whether LLMs could diagnose them using agent traces, though this requires access to the complete codebase and consumes considerable tokens. Zhu *et al.* [31] constructed AgentErrorBench, a dataset of 200 annotated failure trajectories, and proposed AgentDebug, a three-stage framework that traces failures to their root causes. Cemri *et al.* [23] were the first to investigate failure modes in Multi-Agent Systems, and proposed a taxonomy alongside an LLM trained to identify them from benchmark traces [32].

Overall, existing studies either examine bugs across broad domains or focus on agent-building frameworks, leaving a gap in understanding bugs in agentic systems, which persists due to their black-box nature, dynamic behavior, and nascent community. This work addresses both gaps through an extensive study of bugs in LLM agents, with a focus on bug characteristics and practical insights for developers, library maintainers, and researchers, and further evaluates the ability of LLM agents to annotate bugs in their own systems through *BugReAct*, a ReAct-based agent built for this purpose.

III. Empirical Study

As presented in Figure 1, the proposed approach consists of two main steps: collecting and annotating data for the empirical study, and building an LLM agent to evaluate automated bug classification. This section discusses the empirical study conducted to assess bugs encountered by developers during the design of LLM agents. The research team manually analyzed 4,341 bug-related posts and code snippets, from which 1,268 instances were annotated for the study. The empirical study addresses the following research questions.

- **RQ1 (Bug Characterization):** What are the most common bug types, root causes, and effects in developing LLM agents?
- **RQ2 (Bug-Prone Components):** Which component of the LLM agent is more prone to bugs?
- **RQ3 (Bug Mapping):** What is the distribution across bug types, root causes, and frameworks used?
- **RQ4 (Bug Evolution):** How do different types of bugs evolve in frequency over time?

A. Dataset

For the empirical study, we collected a large dataset and annotated it based on bug type, root cause, effect, the LLM agent component involved in bug occurrence, programming language, and development framework. Following previous works [14], [15], two sources were employed in this study, namely the Stack Overflow forum and GitHub, to collect the datasets. In order to test the ability of the LLM agent, we have collected an additional dataset from Hugging Face forums. The process of collecting data from each site is illustrated below.

1) Stack Overflow: We selected Stack Overflow, one of the most popular Q&A sites for developers [46], to collect our dataset. As illustrated in Figure 1, the data collection process consists of three main stages. In the first stage, keyword searching, we gathered posts related to LLM agents by searching with keywords such as `LLM agent`, `LangChain`, `LlamaIndex`, `Semantic Kernel`, `CrewAI`, `AutoGen`, and `OpenAI`, collecting a total of 2929 posts. While each keyword represents a separate framework, `OpenAI` refers to custom agent implementations.

In the second stage, we manually evaluated each post, eliminating those that were irrelevant or consisted only of questions, and posts with insufficient or no code, and removed duplicate entries. We retained only those posts that contained sufficient information about the problem. Finally, we selected 951 posts relevant to our study. In the third step, two PhD students independently analyzed each post and assigned labels. The details of data annotation are mentioned in Section III-A4.

2) Github: We have collected data from two GitHub sources, namely GitHub commits and GitHub Issues, to verify the existence of bugs that we identified in the Stack Overflow dataset. For both GitHub commits and issues, we collected 1891 GitHub repositories utilizing the following libraries: LangChain, LangChain-js, LangGraph, LlamaIndex, CrewAI, AutoGen, and Semantic Kernel. These libraries were selected based on their popularity, as measured by GitHub stars [9].

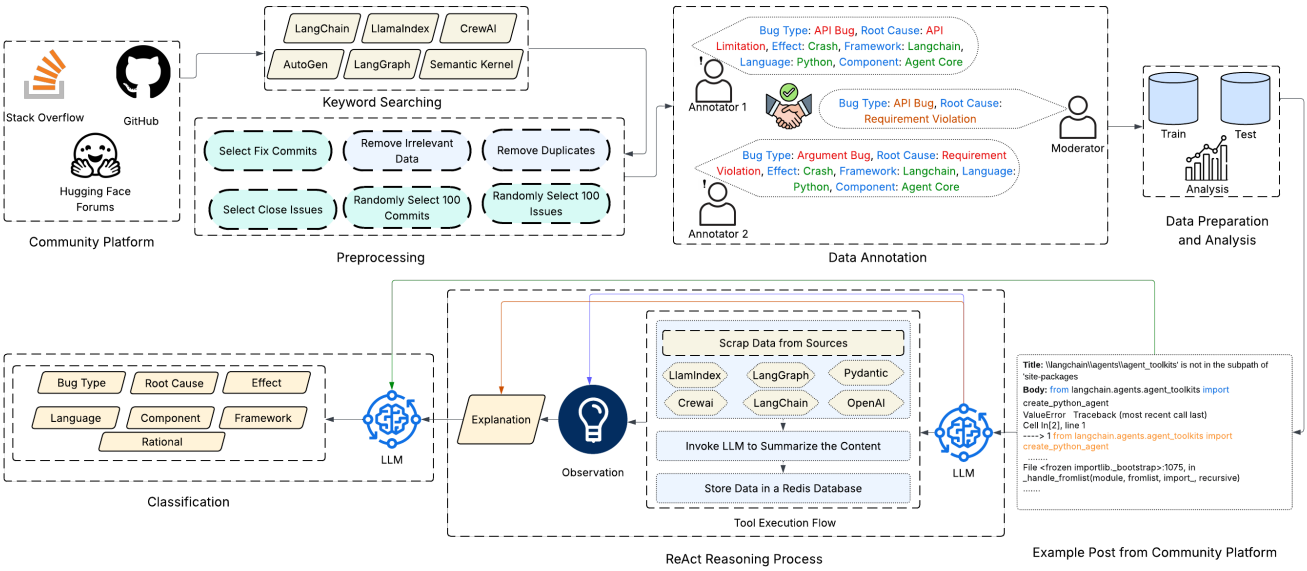


Fig. 1: Workflow of the overall approach.

To identify relevant repositories, we searched GitHub Topics using each library name as the topic keyword, and scraped the resulting repositories using Selenium. We then excluded repositories maintained by the original developers of these libraries to ensure our analysis focused exclusively on bugs in the implementation of agents, rather than bugs within the libraries themselves. While the scraped repositories may include sample projects and course materials, such repositories typically do not contain closed issues or fix commits, and therefore our human annotators confirmed that the selected commits and issues in our dataset do not include repositories of that type. The subsequent steps differ between GitHub commits and issues.

GitHub Commits: After collecting the repositories, we filtered for commits explicitly marked as fix commits (i.e., containing the keyword "fix"). Following prior work [14], we randomly selected 100 commits per library and ensured mutual exclusivity across repositories that share multiple frameworks. Each commit was independently labeled by two annotators using the same bug type, root cause, and effect definitions established for the Stack Overflow dataset, with agreement evaluated at multiple stages. In total, we collected 36, 16, 16, 19, 12, 20, and 23 bugs from LangChain, LangChain-js, LangGraph, LlamaIndex, CrewAI, AutoGen, and Semantic Kernel, respectively.

GitHub Issues: To collect GitHub issues, we have iterated through the repositories and collected all the closed issues. After that, similar to GitHub commits, we randomly chose 100 non-overlapping issues, if a library had more than 100 issues. Subsequently, we filtered out the relevant instances and labeled them following the classification steps defined previously.

3) Hugging Face Forums: Since Hugging Face plays a critical role in building LLM agents, especially with open-source LLMs [47], the Hugging Face forums contain issues related to LLM agents. Therefore, we have selected posts from the forums by searching the keywords: LangChain, LangChain-js, LangGraph, LlamaIndex, CrewAI, AutoGen, and Semantic Kernel. Thereafter, we removed the duplicate posts, and the filtered relevant posts were selected for labeling. The dataset is used for evaluation purposes only. Unlike Stack Overflow and GitHub, which were labeled by two human annotators, this dataset was labeled by one annotator. A second annotator then acted as a reviewer. The reviewer examined both the first annotator's labels and the agent's predicted labels, and independently assigned their own final labels. These final labels could agree with either the human annotator or the agent, or disagree with both.

4) Data Annotation: Since no predefined taxonomy existed for annotating the dataset [48], we adopted methodologies from prior works [10], [15], in which each author independently assigned a card and description to a given post, with final labels defined through open card sorting [49]. The resulting bug types, root causes, and effects are presented in Tables I, II, and III, with associated examples available in [50]. Several labels coincide with categories from prior literature, as discussed in Section III-F alongside their distributions in agentic and non-agentic systems. Beyond bug types, root causes, and effects, we report the programming languages and libraries used to build agents, as well as the agent component that initiated each bug, based on four components from prior literature [10]. Two additional labels are included in the component section, where *Other* indicates the bug originated from a component external to the LLM agent, and *Unknown* indicates the annotators could not determine the responsible component. To understand bug origins and causes, we followed a consistent mechanism across sources. For GitHub commits, the pre-commit version represents the buggy code and the post-commit version reflects the fix. For

TABLE I: Descriptions of Bug Types

Bug Type	Description
Logic Bug (LB)	The bug indicates a lack of logical understanding in the pipeline. This bug includes using a function that does not fit the specific task, missing or incorrectly implementing any code segment, or the absence of proper guarding conditions [14].
Configuration Bug (CB)	This bug indicates any configuration errors, including parameter misconfiguration or environment misconfiguration [33].
Initialization Bug (IB)	Any variable/function may require proper initialization before use. Not initializing the variables/functions correctly before using them or initializing them wrongly (e.g., inside a loop in the wrong way) will cause this error [34].
Argument Bug (AB)	This bug indicates issues with the API signature, such as the API expecting specific arguments while the user provides arguments in an incorrect format, or passes extra or missing parameters. It is worth noting that passing an incorrect value to a function that matches its signature will not trigger this bug [35].
Parsing Bug (PRB)	This LLM-agent-specific bug occurs while parsing the LLM output. Often, the LLM-generated output format does not align with the parser's structure or the user's expectations, triggering the bug. It mostly occurs in the parser of the LLM agent.
Prompting Bug (PPB)	This bug is related to the prompts in the LLM. This includes missing prompt variables/components or invoking the LLM with incorrect instructions.
API Bug (APIB)	This bug is related to the APIs or libraries used to build the agent. Causes include dependency conflicts, incorrect version usage, bugs within the libraries themselves, or trying to use a library without installing it [14].
Reference Bug (RB)	In agentic AI, often different libraries implement modules with similar names. This bug occurs when the user refers to a different, deprecated, or missing module from a specific library. This bug mostly occurs in the import stage.
Availability Bug (AVB)	This bug usually comes from the API or server side, indicating that a particular language model or service is unavailable—either because of a server issue or because the feature has not been released yet.
Model Bug (MB)	This bug is related to the LLM itself and occurs when a user requests a task the LLM cannot perform, such as asking a chat model to generate an image or using a non-functional model to produce a function call.
Resource Limitation Bug (RLB)	This bug is related to the user's local system and its resource usage. It may occur when attempting to run a large LLM with insufficient system resources or limited credits.

TABLE II: Descriptions of Root Causes and Their Subclasses

Root-Cause	Subclass	Definition
AM	Wrong API Context	The API is used in an inappropriate situation, or there is a misunderstanding of its intended purpose. For example, using a translation API to extract sentiment [36].
	Invalid API Arguments	Incorrect types or number of arguments are passed to the API. For instance, passing a string where a JSON object is expected [15].
IMP	Incorrect Value	A valid parameter is passed, but its value is logically wrong, like using a learning rate of 10 instead of 0.01 [37].
	Missing Value	One or more optional parameters are omitted entirely, often leading to unexpected default behavior [37].
IDF	Input Data Format Error	An error that occurs when the data from an external resource provided to the LLM is not in the expected type, structure, or schema. For example, the input text contains a stop sequence that causes the LLM to terminate its response early.
	Output Data Format Error	The expected output format from the LLM is not met. For example, when prompted for structured output, the LLM-generated response may miss required keys or deviate from the format, causing the parser to fail when processing it.
IMCF	Missing Flow	A required logical code segment is not implemented, such as a missing if condition or function call [38].
	Incorrect Flow	This occurs when the logic is present in the code but implemented incorrectly, such as using an if condition that checks the wrong variable [14].
II	Prompt Specification	Errors in constructing the prompt, such as ambiguous language, poor formatting, or missing instruction cues. It includes four issues in the prompt, including missing context, missing specifications, multiple contexts, and unclear instructions [39].
	Prompt Orchestration	Logical flaws in the prompt, such as not passing variables correctly or providing a plain string when a JSON-formatted input is required.
AL		Task failure due to limitations of the API or Library, such as unsupported capabilities or service downtime. Users typically cannot control this [40].
CM		Incorrect usage or selection of components like LLM, tools, or memory modules, resulting in integration failure [41].
RV		Dependency conflicts or missing requirements, such as incompatible library versions or unmet prerequisites [42].
Others		Issues not related to LLM agents, often caused by the user's machine, environment, or external components.

issues, the fix is documented via a pull request or repository owner explanation. For Stack Overflow and HuggingFace posts, verified answers were used, where a verified answer includes an accepted answer, a solution from the original user, a user-confirmed solution in comments, or an annotator-verified solution. When no verified solution was found, external sources were consulted and referenced in the dataset. Each entry also includes a rationale for the selected labels.

Once all the labels are defined using the open card sorting method [49], following a previous study [14], two PhD students independently annotated all four labels (i.e., bug type, root cause, effect, and component). For Stack Overflow, the programming language and framework were identified using tags, while for GitHub, data was collected based on the frameworks. Therefore, these two labels were annotated collaboratively, in contrast to the other four labels, which were annotated independently. Agreement between the two annotators was measured at 5%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, and 100% intervals using Cohen's Kappa coefficient [51], with disagreements addressed after each iteration. When the two annotators failed to reach an agreement, a moderator with expertise in the field adjudicated. At the last iteration, the agreement reached 0.93, 0.95, 0.96, and 1.00 for bug type, root cause, effect, and component, respectively. The agreement scores at different iteration stages for Stack Overflow posts are reported in [50].

At the initial labeling stage, there was slight agreement between the annotators, which increased to a moderate level across all components by 30% of the data. In subsequent iterations, the annotators achieved perfect agreement across all labels for the Stack Overflow dataset. Due to the limited number of samples, agreement for GitHub issues and commits was measured after annotations were completed. Since the GitHub data was annotated after the Stack Overflow data, the level of disagreement was low. For GitHub issues, the kappa

TABLE III: Descriptions of Effects

Effects	Description
Crash	The program throws an error and stops working [14].
Incorrect Output (IO)	LLM generates a complete output, but it does not align with the expected output [14].
Empty Response (ER)	The system does not generate an output at all [14].
Output Dump (OD)	This effect indicates that the system generated the entire output at once instead of streaming it gradually.
Stateless Interaction (SI)	The agent does not remember the past conversations and answers only the current question without understanding the full context.
Partial Output (PO)	The LLM generates incomplete or truncated output [14].
Tool Ignored (TI)	The effect indicates that the system does not invoke tool(s) during the process [43].
Slow Output (SO)	The output is generated slowly [44].
Warning	This means the program does not terminate but instead issues a warning [44].
Hang	The system ceases to respond and requires human intervention to restore functionality [14].
Indeterminate Loop (IL)	The system runs into a loop indefinitely [14].
Resource Overuse (RO)	This effect indicates that the system consumed high resources like RAM [44].
Silent Fail (SF)	This effect illustrates that the agent has failed to perform a task, but did not provide any log that the task failed [45].
Unknown	The post does not mention the effect of the problem [44].

scores for bug type, root cause, and component were 0.9422, 0.7084, and 0.983, respectively, while the effect showed no disagreement. For GitHub commits, no disagreement was recorded. Since some posts may contain multiple bugs, instances with multiple bug types or root causes were annotated multiple times, resulting in 15 such instances and 1,283 total annotations. Lastly, since the Hugging Face forum data was used only for evaluation, one annotator labeled it, agreement was measured against the best LLM, and a second reviewer later checked both.

B. Most common bug type, root cause and effect (RQ1)

While bug distributions vary across platforms, the majority of bug types, root causes, and effects are common to all. Logic Bug is the primary bug type in both Stack Overflow and GitHub, accounting for 22.6% and 30.2% of all bugs, respectively. Table IV presents the distribution of bug types, root causes, and effects.

Although logic bugs also occur in traditional software, existing tools like Pylint [52] cannot be directly applied to LLM agents due to their different reasoning mechanisms and require further adaptation. While parsing and model bugs are less common in GitHub, they appear more frequently in Stack Overflow, likely because GitHub commits reflect already-built systems, whereas parsing and model bugs tend to arise during the initial implementation phase.

Unlike bug types, root causes are more diverse. The primary root causes in Stack Overflow are Incorrect or Missing Control Flow-Incorrect Flow (IMCF.IF) and Requirement Violation. IMCF.IF is common because LLM agent development is a relatively new field, and developers are often unfamiliar with managing the multiple interconnected components that agent systems require. To address this, developers should map out the agent's flow before implementation and use static analysis tools such as Pylint to catch some missing or unhandled conditions. Requirement Violation is also prevalent, as LLM agent development often involves multiple evolving frameworks, making dependency management challenging. This root cause is less common in GitHub, where library versions are often fixed in finished projects. Tools like Poetry [53] can help by locking project dependencies and reducing the risk of conflicting packages.

Crash is the most dominant effect across both platforms, accounting for 60.6% of bugs in Stack Overflow and 47% in GitHub. Unknown is also prominent in GitHub, as the effect is often difficult to determine when not explicitly mentioned in commit descriptions. While some tools localize bugs based on crash reports, their effectiveness in LLM agents remains unexplored [54]. Incorrect output is another common effect across all platforms, while Hang and Resource Overuse are the least frequent.

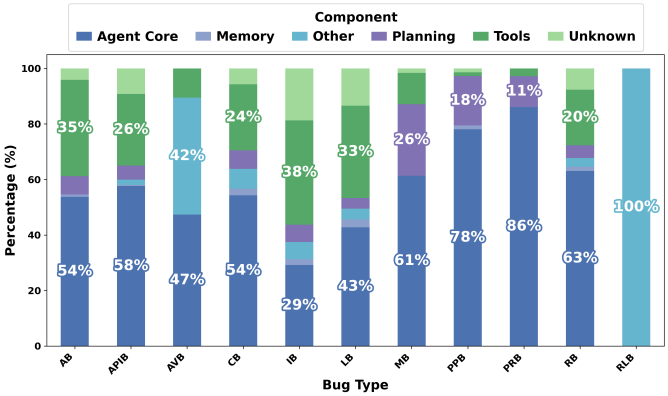


Fig. 2: Component distribution across bug type.

C. Most bug-prone LLM agent component (RQ2)

Although identifying the exact component affected by a bug in GitHub is challenging due to large project sizes, the agent core is the most error-prone among known components. This aligns with Stack Overflow, where the agent core accounts for 57.1% of bugs, followed by tools [50]. These results suggest that prioritizing testing, debugging, and monitoring of the agent core can significantly reduce bug likelihood. The distribution of components across bug types and effects is provided in Figures 2, and 3, respectively, while the distribution of root causes per component is shared in [50]. As shown in Figure 2, the agent core accounts for more than half of the bugs in every bug type, except for Initialization and Resource Limitation bugs. Notably, Resource Limitation bugs do not fall under any agent component, as they are not caused by the LLM agent itself.

According to the distribution of root causes and effects across components, the agent core is the primary component where bugs occur, except for those caused by Incorrect Data

TABLE IV: Distribution of bug types, root causes, and effects across Stack Overflow and GitHub.

Bug Type	Label	LB	APIB	CB	AB	PPB	MB	RB	PRB	RLB	IB	AVB			
	SO	22.6	18.8	18.1	11.2	6.3	5.9	5.4	3.6	3.6	2.6	1.8			
	GH	30.2	16.7	16.7	6.0	5.6	2.3	6.0	0.5	4.2	10.7	0.9			
Root Cause	Label	IMCF.IF	IMCF.MF	IMP.IV	IMP.MV	AM.IAA	AM.WAC	II.PS	II.PO	IDF.ID	IDF.OD	CM	RV	AL	Others
	SO	14.4	8.7	8.6	8.0	9.7	4.8	3.5	2.6	1.3	0.9	5.4	14.1	10.8	7.1
	GH	34.4	6.5	12.1	4.7	8.8	2.8	4.7	0.9	0.5	0.5	2.8	13.0	3.3	5.1
Effect	Label	CR	UN	IO	SF	WR	SO	TI	IL	PO	RO	SI	ER	OD	HG
	SO	60.6	6.6	16.6	1.6	1.0	1.1	2.2	0.9	1.3	0.7	1.6	3.7	1.7	0.2
	GH	47.0	27.0	14.9	5.6	1.4	0.9	0.9	0.5	0.5	0.5	0.5	0.5	0.0	0.0

Flow (Input Data Format Error) and Others. While identifying bug types and root causes from the output alone is difficult, determining effects is comparatively easier. Agents exhibiting indeterminate loops encounter issues in the planning component in 66.7% of cases, whereas stateless interactions stem from bugs in the memory component in 53.33% of cases. Therefore, observed effects can serve as useful indicators for bug localization, allowing developers to narrow their focus to the relevant components and speed up debugging.

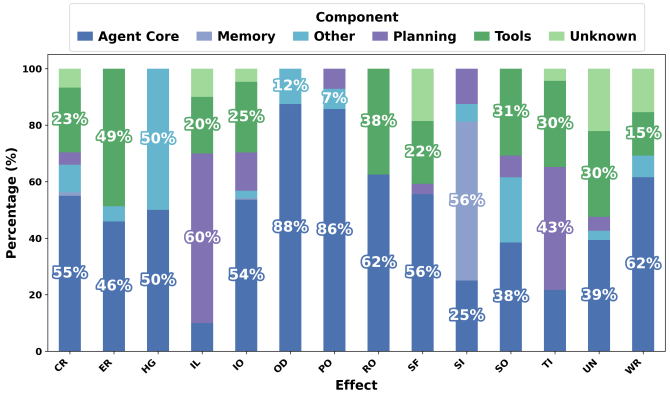


Fig. 3: Component distribution across effects.

D. Distribution between bugs (RQ3)

To answer RQ3, we analyzed the correlation between bug types and their root causes. As shown in Figure 4, most API-related bugs occur due to requirement violations, driven by the rapid release of new library versions and conflicting dependencies across multiple frameworks. LangChain, the most commonly used library [9], is particularly susceptible to such errors. Tools like Poetry [53] can lock library versions, while virtual environments can resolve some conflicts. The second most common cause is API limitations, attributable to rapid library development with limited testing. Argument Bugs, the fourth most common type, predominantly occur due to API misuse, as developers are often unfamiliar with existing method signatures in this emerging field, though their proportion has been decreasing since 2023 as shown in RQ4. Most prompt-related bugs arise from incorrect prompt structure, while fewer stem from prompt logic issues such as incorrectly passing prompt variables. Lastly, resource limitation bugs are categorized under 'Others' root causes, as they are not directly related to agent development.

E. Bug distribution over time (RQ4)

To evaluate the distribution of bugs across different years, we analyzed the six characteristics based on the post date. According to the analysis presented in [50], during 2021 and 2022, which marked the early stage of LLM agents, only a small number of Stack Overflow posts related to bugs in LLM agents were made, totaling fewer than 30 in the collected dataset. At the end of 2022, with the release of libraries for building LLM agents, for instance, LangChain in October 2022, the development of LLM agents surged, resulting in the highest number of bug-related posts on Stack Overflow in 2023. In our dataset, 2024 and 2025 show fewer posts related to bugs in LLM agents. This may reflect fewer development challenges, a slight drop in development activity, framework changes that reduced errors, or tools like ChatGPT helping developers resolve bugs on their own. Further investigation is needed to understand this trend. The number of posts in 2026 does not reflect the entire year, as data collection stopped after March 2025.

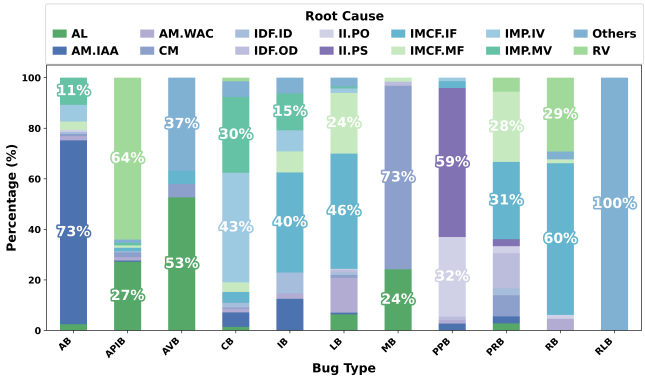


Fig. 4: Distribution of root cause across bug types.

As shared in [50], the proportion of reported bugs in JavaScript has been decreasing since 2023, with 14.56% of bugs reported in 2023 coming from JavaScript, dropping to 7.30% in 2025. In contrast, the proportion of bugs reported in C# has been increasing steadily during the same period, rising from 2.63% to 5.11%. The trend may suggest that developers use JavaScript less for building agents, or that they encounter fewer bugs when doing so, though further investigation is needed to understand the trend.

Most major libraries, such as LangChain, Autogen, and CrewAI, were released at the end of 2022, after which developers began adopting them over custom-made agents. As shared in [50], LangChain remains the most popular framework.

Semantic Kernel usage grew from 0.95% in 2023 to 5.84% in 2025, and LangGraph gained significant traction as multi-agent systems became more complex, with its share rising from 1.64% in 2024 to 5.84% in 2025. This trend highlights the need for automatic code generation and optimization tools for LLM agents, similar to Auto-Keras for DNNs [55].

Since LangChain is the most popular framework for building LLM agents [9], we investigated the common root causes of bugs in LangChain-based agents. As detailed in [50], API limitations and API misuse accounted for 25.22% of bugs in 2023, rising to 29.86% in 2025. A similar trend is observed in CrewAI [50], where this proportion increased from 22.23% in 2024 to 60% in 2025. An increasing number of API limitation bugs is also reported in LangChain-js, LlamaIndex, Semantic Kernel, and Autogen, driven by the rapid evolution of these libraries. Library maintainers should prioritize rigorous documentation before releasing new versions. To address API misuse, researchers should treat agentic libraries and general-purpose libraries as distinct categories, similar to efforts in the deep learning field [56].

F. Cross-System Bugs Comparison

Our study identifies several bug types, root causes, and effects that also appear in other software systems, such as Logic and Initialization Bugs, API Misuse, and Crash. However, we also identify categories unique to agentic systems, including Parsing, Reference, Availability, Model, and Resource Limitation Bugs, as well as novel root causes such as Prompt Orchestration. These are rare in traditional software due to the distinct characteristics of agentic systems. For instance, LLMs produce dynamic output that demands flexible parsers, and frequent library reorganization causes outdated references and import errors. Availability bugs arise because failures in external LLMs or tools can bring down the entire agent. Model bugs are amplified by the diversity of LLM variants that makes it difficult to select the appropriate model for a task. Resource limitation bugs are more prevalent due to the specialized computational demands of LLMs.

Although some bug categories overlap with those in other software fields, their distribution differs significantly in agentic systems. In deep learning compiler systems, 9.78% of bugs stem from API misuse [57], and a study on open-source C programs reports 17.05%, compared to 14.5% and 11.6% in agentic systems on Stack Overflow and GitHub, respectively. Similarly, data format bugs account for 25% of bugs in deep learning systems [58] but only 2.2% in agentic systems, as agents rely on pre-trained models rather than fine-tuning or training from scratch, which reduces dependencies on input datasets and data formatting. Some similarities in bug distribution are also observed; for instance, API bugs in PyTorch-based deep learning systems account for 16% [14], which is consistent with the distribution observed in this study.

G. Actionable Insights

This section discusses the implications from our study (RQ1–RQ4) and provides recommendations for developers, library maintainers, and researchers to minimize bugs in LLM agents. A summary of the insights is shared in [50].

1) Developers: From RQ3, requirement violation is the most common cause of API bugs, affecting 61.88% of this type. Since new library releases often introduce conflicting requirements in this rapidly evolving field, to address some of these issues, developers can leverage dependency and packaging tools such as Poetry [53], which defines a project's dependencies in a single file and ensures consistent package versions. Additionally, argument-related bugs, largely caused by misaligned parameters, have been decreasing since 2023, though developers can further address this by integrating type-checking tools to ensure correct method invocations [59]. Lastly, API limitation bugs account for 10.8% of Stack Overflow bugs and have been increasing over the past three years across most libraries (RQ4). While identifying such bugs can be challenging for small teams, reviewing a library's open issues before adoption is a practical approach to gain awareness of potential challenges.

2) Library Maintainers: As LLM agent libraries grow rapidly, new versions often conflict with existing dependencies. For instance, LlamaIndex released over 140 versions in 2024 [60]. Therefore, when releasing a new version, library maintainers should not only document newly added features but also specify any underlying version changes, so developers can anticipate potential conflicts during installation. To further mitigate the increasing number of API limitation bugs highlighted in RQ4, library maintainers should conduct more extensive testing before new releases.

3) Researchers: As API limitations are common beyond LLM agents, multiple Software Composition Analysis (SCA) tools exist to report known vulnerabilities, licensing issues, and outdated components [61]. OSV-Scanner [62], developed by Google, is one popular tool in this domain. It queries the OSV database [63] to identify vulnerabilities. However, while the database includes some known vulnerabilities of LLM agent libraries such as LangChain, OSV-Scanner often fails to capture risks specific to these libraries. To the best of our knowledge, existing SCA tools are not well-suited for LLM agent libraries. Researchers could extend existing SCA tools or develop new ones that aggregate known issues from community platforms such as Stack Overflow, Hugging Face forums, OpenAI Discussions, or Reddit, to provide developers with a more comprehensive understanding of potential vulnerabilities before adopting a library.

IV. LLM Agent for Labeling Bugs

This section describes the LLM agent designed for annotating bugs. As shown in Figure 1, the agent consists of two main components. The first component is a *ReAct* agent that leverages different tools to reason about where the bug occurred. The second component takes this reasoning and maps it to the appropriate class labels. The subsequent sections explain the two components of the agent in detail, along with an example explaining the classification process.

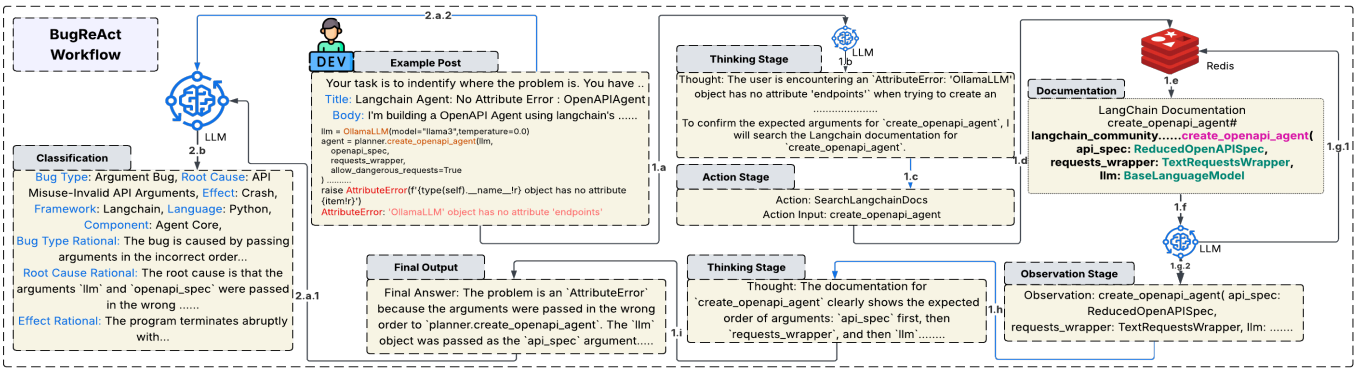


Fig. 5: The flow of automatic annotation for a Stack Overflow post [64] through *BugReAct*, equipped with Gemini 2.5 Flash.

A. *BugReAct Agent*

To annotate bugs in LLM agents, we built a *BugReAct* agent [65]. Although multiple agent types exist, including Reflexion [66], function-calling [67], and ExpeL agents [68], *ReAct* agents stand out for their explicit thought-action-observation loop, which simplifies tool integration and debugging [69]. The proposed agent is equipped with 10 tools that scrape documentation for eight libraries (LangChain, LangChain-js, LangGraph, Pydantic, CrewAI, LlamaIndex, Semantic Kernel, and Autogen) and two community forums (the OpenAI developer community and GitHub Discussions, the latter covering all eight frameworks). Since documentation and forum searches can yield irrelevant results that overflow the context window, a summarization module is introduced after each search to condense retrieved content. To further improve efficiency, an in-memory database (Redis [70]) is integrated to avoid reprocessing previously searched content, reducing retrieval delay and improving agent speed. The agent can invoke any tool multiple times, with each output treated as an observation that guides subsequent actions, until enough information is gathered to produce a final output with both a bug explanation and the reasoning behind its occurrence.

B. *Classification*

This component takes the explanation generated by the *BugReAct* agent along with the post or code snippet and passes them through an LLM for classification. The LLM is provided with class labels and definitions via the prompt and assigns one label per category, along with rationales for bug type, root cause, and effect. Pydantic enforces structured output, and for LLMs that do not natively support this, strict prompting is applied to ensure the required format.

C. *Workflow of BugReAct with an Illustrative Example*

Figure 5 illustrates the decision-making workflow of a Stack Overflow post [64] processed by *BugReAct*. The process begins when the post is wrapped in a prompt and passed to the LLM agent (1.a), which analyzes the input and formulates an action plan (1.b). The agent then transitions to the action stage, where it determines that consulting the LangChain documentation is needed (1.c). The documentation tool is invoked with a specific query and first checks whether the keyword exists in the database (1.d). If not, it queries the LangChain documentation directly (1.e). Since retrieved content may be irrelevant or exceed the context window, it is passed through a summarization step (1.f). The summarized output is stored in Redis for future use (1.g.1) and returned to the agent (1.g.2). The agent interprets this as an observation and proceeds to the thought stage (1.h). If the output is insufficient, the tool is re-invoked to gather more information (1.i). Once adequate evidence is collected, the agent produces a final decision with an explanation of the bug's underlying cause, concluding the first stage of *BugReAct*.

The second stage begins when the final output from Stage 1 (2.a.1) and the original input (2.a.2) are passed to the LLM, along with a predefined set of labels and their definitions. Based on these inputs, the model selects six labels describing the bug's characteristics, accompanied by rationales for bug type, root cause, and effect (2.b). The predicted labels are then compared against the human-annotated benchmark to evaluate model performance.

V. Evaluation

We implemented *BugReAct* in Python, using LangChain as the primary agent framework, alongside Pydantic for structured output, Selenium for web scraping, and Redis for in-memory storage. Experiments were run on a MacBook M3 with 16 GB RAM, with LLMs invoked via the OpenAI, OpenRouter, and Claude APIs. The codebase follows a modular structure with proper exception handling. We evaluated *BugReAct* based on the following research questions:

- **RQ5 (Effectiveness)** How effective is *BugReAct* in characterizing bugs in LLM agents?
- **RQ6 (Reliability)** How accurate is *BugReAct* in labeling LLM bugs compared to human annotators?
- **RQ7 (Efficiency & Cost)** How efficient and cost-effective is *BugReAct* in characterizing bug types, root causes, and effects?

A. *Effectiveness*

To evaluate *BugReAct*, we tested it on an annotated Stack Overflow and GitHub dataset using three powerful LLMs, namely Gemini 2.5 Flash [71], GPT o3 mini [72], and Claude

Sonnet 4 [73]. These models were selected for their state-of-the-art reasoning capabilities, compatibility with *BugReAct* agents, and diversity across providers. Since the collected dataset is highly imbalanced, accuracy alone can yield a biased evaluation. Therefore, we report the F1-score, which combines precision and recall for a more balanced assessment. For a deeper analysis of the Stack Overflow dataset, we also evaluate *BugReAct* with and without the accepted answer. Figure 6 presents the F1-scores of the three LLMs on both datasets. Here BT: Bug Type, RC: Root Cause, Ef: Effect, Ln: Language, Cp: Component, Fr: Framework, SO: Stack Overflow, GH: GitHub, WO: without accepted answer, WA: with accepted answer, Iss: GitHub issues, Com: GitHub commits, gm: Gemini, gp: GPT, cl: Claude.

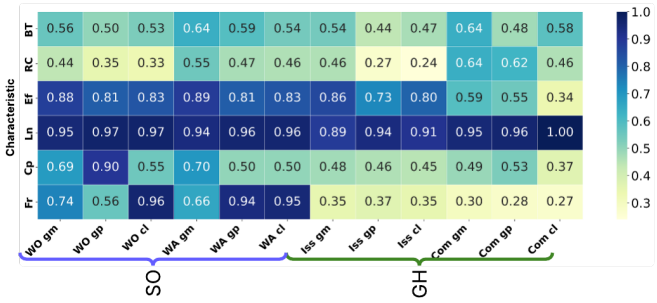


Fig. 6: F1-score of different models in different datasets.

Gemini 2.5 Flash consistently achieves the best performance across both datasets, followed by GPT o3 mini and then Claude Sonnet 4. GPT o3 mini tends to produce shorter, more compact outputs [74], but often skips necessary tool calls to verify its knowledge, unlike Gemini 2.5 Flash, which actively invokes tools to confirm specific details. Claude Sonnet 4, despite being the latest in its family, frequently over-invokes tools and struggles to converge on a correct conclusion. Our pilot study with Claude Opus 4 revealed a 4.36% lower F1-score compared to Claude Sonnet 4.

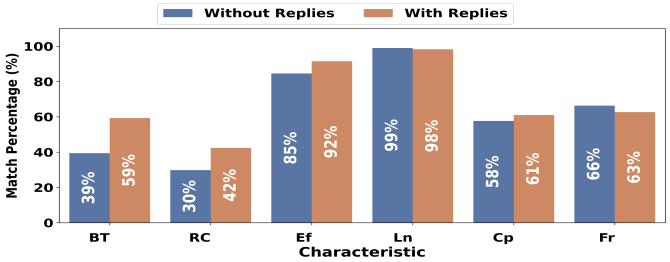


Fig. 7: Comparison of *BugReAct*'s match with human annotators, with and without replies.

B. Reliability

For reliability evaluation, we compare *BugReAct* predictions with those of a human annotator on the Hugging Face Forums dataset. Figure 7 shows this comparison for Gemini 2.5 Flash and GPT o3 mini. *BugReAct* matches only 39.4% of human annotations when provided with just the title and body of a post. This match rises to 59.3% when replies, specifically those marked by the human annotator as solutions, are included. Agreement is higher in critical components such as root cause, effect, and LLM agent component, while less critical aspects such as language and framework show slight decreases when the accepted answer is provided. Although performance dropped for GPT o3 mini and Claude Sonnet 4, *BugReAct* demonstrates strong agreement with the human annotator, particularly when given the full post context with replies. Whether the agent can consistently provide annotations with clear rationales, comparable to those of human annotators, remains an open question for future research. To understand our solution's failure points, we manually analyzed the instances where *BugReAct* failed to correctly annotate bugs. In these cases, the tool is invoked with incorrect input, such as wrong search terms, which leads to a wrong assessment of the bug and, in turn, wrong labels.

To further evaluate effectiveness, a second human annotator reviewed both the human-generated and *BugReAct*-generated labels and produced an independent label. The agreement table shared in [50] presents the agreement rates between the reviewer, the original annotator, and *BugReAct*. Since Gemini 2.5 Flash achieved the best performance, only this model was used. The reviewer agreed with the human annotator in all cases except bug type (93.33%) and root cause (97.14%). On HuggingFace forum posts, *BugReAct* achieved notable performance in effect annotation but lower performance in root cause annotation.

TABLE V: Effectiveness (seconds) and cost ($) of different LLMs across different platforms.

Dataset and Setting		Metric	Gemini	GPT	Claude
Stack Overflow	Without Answer	Time	34.00	19.09	125.93
		Cost	0.016	0.017	0.049
	With Answer	Time	7.82	11.84	55.90
		Cost	0.006	0.005	0.039
GitHub	Issues	Time	22.31	17.28	118.18
		Cost	0.008	0.014	0.025
	Commits	Time	18.01	15.41	97.36
		Cost	0.013	0.002	0.013
Hugging Face	Without Answer	Time	20.19	25.78	100.38
		Cost	0.021	0.024	0.041
	With Answer	Time	9.39	14.69	75.67
		Cost	0.007	0.010	0.015

C. Efficiency and Cost

To evaluate the cost and efficiency of *BugReAct*, we recorded the API cost and time taken per post or commit across the three platforms, as presented in Table V. Claude Sonnet 4 consumed the most time and incurred the highest cost to reach a solution. Gemini 2.5 Flash required slightly more time than GPT o3 Mini on average, but unlike GPT o3 mini, it invoked multiple tools before arriving at a solution. When provided with a community answer, the agent made fewer tool calls and converged faster, thereby reducing the overall cost. We also conducted a pilot study with Claude Opus 4 on 110 Stack Overflow posts without accepted answers, which averaged 115.4 seconds and 0.455 USD per post.

D. Comparison with Baselines

To compare *BugReAct* with existing solutions, we evaluated four architectural settings covering both encoder-based and decoder-based LLMs.

Encoder: In this setting, input text is fed into an encoder transformer that generates a representation passed to a classification head. While fast and effective in specific

cases [75], it requires labeled training data. We investigated CodeBERT-base [76] and CodeT5-base [77], both known for strong performance in code-related tasks [78], fine-tuning on 80% of the data and testing on the remaining 20%.

Zero-shot: The LLM receives the input text and possible labels and selects one without additional information [79]. This setting uses the same LLMs as *BugReAct*.

One-shot: Similar to zero-shot, but with one example per category. The shortest example by character length was selected to fit within the prompt window. For Stack Overflow and GitHub issues, even the shortest examples exceeded the contest window limit. To address this, the same LLM used for classification generated a concise summary of each example, which was included in the prompt instead of the full text.

Tool-less ReAct: This setting mirrors *BugReAct* but disables tool access. By returning a fixed "No results found" response whenever the agent attempts to invoke a tool, we establish a baseline to measure the performance gain specifically attributed to our tool integration.

TABLE VI: LLM comparison on Stack Overflow and GitHub tasks (W/O: Without). Complete table shared in [50].

Approach	Model	Category	Stack Overflow		GitHub	
			W/O Ans.	With Ans.	Commit	Issues
Encoder	CodeBERT	Bug Type	0.19	0.29	0.41	0.33
		Root Cause	0.17	0.19	0.35	0.29
		Effect	0.57	0.61	0.35	0.64
	CodeT5	Bug Type	0.24	0.24	0.47	0.33
		Root Cause	0.17	0.10	0.40	0.23
		Effect	0.55	0.58	0.36	0.64
Zero-shot	Gemini	Bug Type	0.21	0.22	0.23	0.21
		Root Cause	0.13	0.18	0.11	0.24
		Effect	0.35	0.29	0.18	0.40
One-shot	Gemini	Bug Type	0.24	0.17	0.16	0.37
		Root Cause	0.18	0.18	0.14	0.22
		Effect	0.50	0.56	0.11	0.37
Tool-less ReAct	Gemini	Bug Type	0.30	0.20	0.19	0.30
		Root Cause	0.14	0.18	0.22	0.27
		Effect	0.45	0.24	0.24	0.37

E. Results Analysis and Discussion

Table VI presents the F1-scores of all baseline architectures. For zero-shot, one-shot, and ReAct settings, only results of the best-performing LLM (Gemini 2.5 Flash) are shown for the critical components (bug type, root cause, and effect), with complete results shared in [50]. Among the three non-trivial categories, *BugReAct* outperformed all baselines except in the effect category for GitHub commits, and outperformed most baselines on the trivial categories. The Tool-less ReAct agent achieved an average F1-score of only 25.71% on non-trivial categories, while *BugReAct* achieved 64.08% using Gemini 2.5 Flash, demonstrating the significant impact of the developed tools. Encoder-based solutions also performed well. The one-shot approach underperformed zero-shot, likely because the example was selected based on length rather than category representativeness, which may have misled the LLM.

While *BugReAct* outperformed all baselines, it has several limitations. The system depends on external sources, and their unavailability may significantly affect performance. Web scraping also introduces latency due to factors such as internet speed. Furthermore, the agent was evaluated on posts and small code snippets, and may require architectural adjustments to handle larger codebases. Overall, *BugReAct* demonstrated strong performance in annotating LLM agent bugs. While prior studies reported limited self-healing ability in LLM agents [11], this work shows that agents can identify and annotate bugs in agentic systems when appropriately designed. Although *BugReAct* targets bug annotation, it can be adapted for bug fixing by replacing the classification head with a fixation head, an adaptation future studies can explore.

VI. Threats to Validity

Internal Threat: One potential internal threat is bug classification. While no predefined taxonomy was used, some labels were adapted from prior literature and new ones were introduced where none existed. To mitigate the threat of potential bias in the annotation, two researchers independently annotated bugs and resolved disagreements through discussion with a knowledgeable moderator. We also provide rationales for each label across bug type, root cause, and effect. Another threat is the annotation of Stack Overflow and Hugging Face Forums posts without accepted answers. Posts with a verified solution were marked accordingly, and for others, external resources were referenced to support annotation decisions. Although both annotators verified these resources, minor subjectivity may remain since solutions were not confirmed by the users. Lastly, we acknowledge that the proposed taxonomy, though grounded in current literature and empirical observations, may evolve as LLM agent architectures advance.

External Threat: One potential threat is dataset inclusiveness, as libraries were selected based on popularity using GitHub stars [9] and related Stack Overflow posts. Not all LLM agent libraries are represented, particularly those with limited adoption, and since the dataset is derived from community-reported bugs, popular frameworks are inherently overrepresented. Lastly, *BugReAct* invokes 10 tools that scrape specific web pages, and any changes to these pages after April 2026 may affect tool calls and require adjustments to the scraping.

VII. Conclusions and Future Work

In recent years, the use of LLM agents has grown significantly, while developers have faced critical challenges in building and maintaining these systems due to the novelty of the field and limited knowledge about the types of bugs that occur. To address this issue, we conducted the first comprehensive study of characterizing bugs in LLM agents. Our empirical study provides a deeper understanding of bugs across three community forums, namely Stack Overflow, GitHub, and Hugging Face discussions. Based on our findings, we provide actionable takeaways for developers, library maintainers, and researchers to improve practices and accommodate necessary changes. Additionally, we developed an LLM-based *ReAct* agent named *BugReAct* to automatically annotate bugs from community platforms and evaluate the effectiveness of LLM agents in identifying and annotating bugs. Our study highlights the potential of LLMs, particularly Gemini 2.5 Flash, for this task at low cost. Future work should explore LLMs' capabilities in fixing bugs in LLM agents and perform deeper analyses to achieve human-level performance in both annotating and resolving bugs.

Data Availability

Our benchmarks, results, findings, and implementation code of *BugReAct* are available in this repository [50].

Acknowledgment

This work was supported by the National Science Foundation (NSF) under Grant No. IIS-2419883. Any opinions, findings, and conclusions or recommendations expressed in this work are those of the author(s) and do not necessarily reflect the views of the NSF.

References

[1] K. Huang, "The genesis and evolution of ai agents," in *Agentic AI: Theories and Practices*. Springer, 2025, pp. 1–22.

[2] J. Huang, K. Huang, and C. Hughes, "Ai agents in offensive security," in *Agentic AI: Theories and Practices*. Springer, 2025, pp. 167–205.

[3] K. Huang, "Ai agents in healthcare," in *Agentic AI: Theories and Practices*. Springer, 2025, pp. 303–321.

[4] J. Huang and K. Huang, *AI Agents in Robotics*. Cham: Springer Nature Switzerland, 2025, pp. 323–368. [Online]. Available: https://doi.org/10.1007/978-3-031-90026-6_11

[5] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *CoRR*, 2024.

[6] S. Hosseini and H. Seilani, "The role of agentic ai in shaping a smart future: A systematic review," *Array*, p. 100399, 2025.

[7] J. Kim, D. Kim, and Y. Yang, "Learning to correct for QA reasoning with black-box LLMs," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8916–8937. [Online]. Available: https://aclanthology.org/2024.emnlp-main.504/

[8] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of chatgpt in code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–28, 2025.

[9] kaushikb11, "awesome-llm-agents: A curated list of awesome llm agents frameworks," https://github.com/kaushikb11/awesome-llm-agents, 2025, gitHub repository, last updated: 2025-09-07.

[10] K. Ning, J. Chen, J. Zhang, W. Li, Z. Wang, Y. Feng, W. Zhang, and Z. Zheng, "Defining and detecting the defects of large language model-based autonomous agents," *IEEE Transactions on Software Engineering*, vol. 52, no. 3, pp. 1074–1093, 2026.

[11] A. W. Rahardja, J. Liu, W. Chen, Z. Chen, and Y. Lou, "Can agents fix agent issues?" *arXiv preprint arXiv:2505.20749*, 2025.

[12] L. Zhu, X. Wang, and X. Wang, "Judgelm: Fine-tuned large language models are scalable judges," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 51 257–51 296.

[13] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, "Llms-as-judges: a comprehensive survey on llm-based evaluation methods," *arXiv preprint arXiv:2412.05579*, 2024.

[14] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 510–520.

[15] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 1110–1121.

[16] F. Thung, S. Wang, D. Lo, and L. Jiang, "An empirical study of bugs in machine learning systems," in *2012 IEEE 23rd International Symposium on Software Reliability Engineering*. IEEE, 2012, pp. 271–280.

[17] S. Jahan, S. S. Rajput, T. Sharma, and M. M. Rahman, "Taxonomy of faults in attention-based neural networks," *arXiv preprint arXiv:2508.04925*, 2025.

[18] D.-G. Thomas, M. Biagiola, N. Humbatova, M. Wardat, G. Jahangirova, H. Rajan, and P. Tonella, "muprl: A mutation testing pipeline for deep reinforcement learning based on real faults," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2238–2250.

[19] W. Jiang, X. Zhang, X. Xie, J. Yu, Y. Zhi, S. Ma, and C. Shen, "The foundation cracks: A comprehensive study on bugs and testing practices in llm libraries," *arXiv preprint arXiv:2506.12320*, 2025.

[20] J. Han, G. Wang, J. Liu, L. Bao, X. Hu, J. Wei, and S. Deng, "A comprehensive study of bug characteristics on foundation language models," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 2025, pp. 257–268.

[21] Z. Xue, Y. Zhao, S. Wang, K. Chen, and H. Wang, "A characterization study of bugs in llm agent workflow orchestration frameworks," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 3369–3380.

[22] X. Zhang, H. Zhang, and S. H. Tan, "Dissecting bug triggers and failure modes in modern agentic frameworks: An empirical study," *arXiv preprint arXiv:2604.08906*, 2026.

[23] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. A. Zaharia, J. Gonzalez, and I. Stoica, "Why do multi-agent llm systems fail?" in *Advances in Neural Information Processing Systems*, vol. 38, Main Conference. Curran Associates, Inc., 2025.

[24] N. Islam, M. A. Raza, and M. Wardat, "Selfheal: Empirical fix pattern analysis and bug repair in llm agents," *arXiv preprint arXiv:2604.17699*, 2026.

[25] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying llm-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025.

[26] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604.

[27] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 50 528–50 652, 2024.

[28] W. Epperson, G. Bansal, V. C. Dibia, A. Fourney, J. Gerrits, E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent ai systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–15.

[29] C. Zhu, S. Hong, J. Wu, K. Chawla, Y. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "RAFFLES: Reasoning-based attribution of faults for LLM systems," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 7659–7688. [Online]. Available: https://aclanthology.org/2026.eacl-long.359/

[30] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "Trail: Trace reasoning and agentic issue localization," *arXiv preprint arXiv:2505.08638*, 2025.

[31] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang *et al.*, "Where llm agents fail and how they can learn from failures," *arXiv preprint arXiv:2509.25370*, 2025.

[32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *ICLR*, 2024.

[33] X. Han and T. Yu, "An empirical study on performance bugs for highly configurable software systems," in *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2016, pp. 1–10.

[34] Y. Zhai, Y. Hao, H. Zhang, D. Wang, C. Song, Z. Qian, M. Lesani, S. V. Krishnamurthy, and P. Yu, "Ubitect: a precise and scalable method to detect use-before-initialization bugs in linux kernel," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 221–232.

[35] A. Rice, E. Aftandilian, C. Jaspan, E. Johnston, M. Pradel, and Y. Arroyo-Paredes, "Detecting argument selection defects," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–22, 2017.

[36] K. Ferles, J. Stephens, and I. Dillig, "Verifying correct usage of context-free api protocols," *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–30, 2021.

[37] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, and R. Jabbarvand, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[38] L. Jiang and Z. Su, "Context-aware statistical debugging: from bug predictors to faulty control flow paths," in *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering*, 2007, pp. 184–193.

[39] R. Ehsani, S. Pathak, and P. Chatterjee, "Towards detecting prompt knowledge gaps for improved llm-guided issue resolution," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 699–711.

[40] X. Li, J. Jiang, S. Benton, Y. Xiong, and L. Zhang, "A large-scale study on api misuses in the wild," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 241–252.

[41] J. Wang, G. Xiao, S. Zhang, H. Lei, Y. Liu, and Y. Sui, "Compatibility issues in deep learning systems: Problems and opportunities," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 476–488.

[42] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.

[43] C. Winston and R. Just, "A taxonomy of failures in tool-augmented llms," in *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 2025, pp. 125–135.

[44] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empirical software engineering*, vol. 19, no. 6, pp. 1665–1705, 2014.

[45] C. Lou, Y. Jing, and P. Huang, "Demystifying and checking silent semantic violations in large distributed systems," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 91–107.

[46] I. Moutidis and H. T. Williams, "Community evolution on stack overflow," *Plos one*, vol. 16, no. 6, p. e0253010, 2021.

[47] Z. Hussain, M. Binz, R. Mata, and D. U. Wulff, "A tutorial on open-source large language models for behavioral science," *Behavior Research Methods*, vol. 56, no. 8, pp. 8214–8237, 2024.

[48] R. Nistor and L. Applis, "What about haskell bugs? adapting bug taxonomies to haskell's features and community," in *Proceedings of the 36th Symposium on Implementation and Application of Functional Languages*, 2025, pp. 38–50.

[49] F. Zampetti, G. Fucci, A. Serebrenik, and M. Di Penta, "Self-admitted technical debt practices: a comparison between industry and open-source," *Empirical Software Engineering*, vol. 26, no. 6, p. 131, 2021.

[50] N. Islam, R. S. Ayon, D. G. Thomas, S. Ahmed, and M. Wardat, "When agents fail: A comprehensive study of bugs in llm agents with automated labeling," Aug. 2026. [Online]. Available: https://doi.org/10.5281/zenodo.21985724

[51] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[52] "Pylint," https://pypi.org/project/pylint/, Accessed: 2026-04-24.

[53] "Poetry – python dependency management and packaging tool," https://python-poetry.org, Accessed: 2026-04-24.

[54] M. Medeiros, U. Kulesza, R. Coelho, R. Bonifácio, C. Treude, and E. A. Barbosa, "The impact of bug localization based on crash report mining: A developers' perspective," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024, pp. 13–24.

[55] H. Jin, Q. Song, and X. Hu, "Auto-keras: An efficient neural architecture search system," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019.

[56] D. Yang, K. Liu, Y. Lei, L. Li, H. Xie, C. Liu, Z. Wang, X. Mao, and T. F. Bissyandé, "Demystifying api misuses in deep learning applications," *Empirical Software Engineering*, vol. 29, no. 2, p. 45, 2024.

[57] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proceedings of the 29th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2021, pp. 968–980.

[58] M. B. Shah, M. M. Rahman, and F. Khomh, "Towards understanding the impact of data bugs on deep learning models in software engineering," *Empirical Software Engineering*, vol. 30, no. 6, p. 168, 2025.

[59] W. Oh and H. Oh, "Towards effective static type-error detection for python," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1808–1820.

[60] L. Contributors, "Llama index python package," https://pypi.org/project/llama-index/#history, PyPI, 2026, accessed: 2026-04-07.

[61] P. Sharma, Z. Shi, Ş. Şimşek, D. Starobinski, and D. S. Medina, "Understanding similarities and differences between software composition analysis tools," *IEEE Security & Privacy*, 2024.

[62] Google OSV Scanner, "Osv scanner," https://github.com/google/osv-scanner, 2025, accessed: 2026-04-24.

[63] "Osv – open source vulnerabilities database," https://osv.dev, Accessed: 2026-04-24.

[64] Badhusha. (2025) Langchain agent: No attribute error : Openapiagent. Accessed: 2026-04-24. [Online]. Available: https://stackoverflow.com/questions/79565168

[65] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[66] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf

[67] I. Abdelaziz, K. Basu, M. Agarwal, S. Kumaravel, M. Stallone, R. Panda, Y. Rizk, G. P. S. Bhargav, M. Crouse, C. Gunasekara, S. Ikbal, S. Joshi, H. Karanam, V. Kumar, A. Munawar, S. Neelam, D. Raghu, U. Sharma, A. M. Soria, D. Sreedhar, P. Venkateswaran, M. Unuvar, D. D. Cox, S. Roukos, L. A. Lastras, and P. Kapanipathi, "Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, US: Association for Computational Linguistics, Nov. 2024, pp. 1131–1139. [Online]. Available: https://aclanthology.org/2024.emnlp-industry.85/

[68] Y. Deng, S. Fan, N. Wang, X. Zhao, and S.-K. Ng, "AgentPro: Enhancing LLM agents with automated process supervision," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 9981–10 006. [Online]. Available: https://aclanthology.org/2025.emnlp-main.506/

[69] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, pp. 19 632–19 642, Mar. 2024. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/29936

[70] Redis, "Redis — the real-time data platform," https://redis.io/, 2025, accessed: 2025-09-10.

[71] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[72] OpenAI, "Introducing openai o3 and o4-mini," https://openai.com/index/introducing-o3-and-o4-mini/, 2025, accessed: 2026-04-24.

[73] Anthropic, "Claude sonnet 4," https://www.anthropic.com/claude/sonnet, 2025, accessed: 2026-04-24.

[74] M. Ballon, A. Algaba, and V. Ginis, "The relationship between reasoning and performance in large language models–o3 (mini) thinks harder, not longer," *Scientific Reports*, 2026.

[75] R. G. Hussain, K.-C. Yow, and M. Gori, "Leveraging an enhanced codebert-based model for multiclass software defect prediction via defect classification," *IEEE access*, 2025.

[76] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," pp. 1536–1547, Nov. 2020.

[77] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," pp. 8696–8708, Nov. 2021. [Online]. Available: https://aclanthology.org/2021.emnlp-main.685/

[78] D. P. Ghale and M. Dabbagh, "Automated code comments generation using large language models: Empirical evaluation of t5 and bart," *IEEE Access*, 2025.

[79] Y. Wang, W. Wang, Q. Chen, K. Huang, A. Nguyen, and S. De, "Prompt-based zero-shot text classification with conceptual knowledge," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, 2023, pp. 30–38.